

1 Problembeskrivelse

Jyllands-Posten anvender i stigende grad kampagnebaserede abonnementer til at tiltrække nye digitale kunder. Selvom disse kampagner genererer en høj tilgang af abonnenter, oplever virksomheden en markant churn blandt kampagneabonnenter kort tid efter kampagneperiodens udløb.

Den høje churn-rate udfordrer abonnementsforretningens langsigtede bæredygtighed og vanskeliggør en effektiv prioritering af fastholdelses- og genkøbsindsatser. Jyllands-Posten råder over omfattende data om abonnenternes adfærd, historik og engagement i kampagneperioden, men disse data anvendes i begrænset omfang til systematisk og forklarlig forudsigelse af churn.

Der er derfor behov for at undersøge, i hvilken grad Machine Learning-modeller kan anvendes til at forudsige churn blandt kampagneabonnenter baseret på deres adfærd og historik, samt hvordan disse prædiktioner kan gøres forståelige og handlingsorienterede for forretningen.

For at understøtte Jyllands-Posten anvendelse skal modelresultater og centrale churn-drivere visualiseres i en intuitiv dashboard-løsning, der kan bruges af ikke-tekniske interessenter til at identificere risikokunder, vigtige mønstre og relevante indsatsområder. Både i relation til fastholdelse og planlægning af fremtidige kampagner.

1.1 Afgrænsning

I projektet fokuseres der udelukkende på abonnenternes adfærd og historik i selve kampagneperioden. Analysen baseres på data, der er tilgængelige fra købstidspunktet og frem til kampagneperiodens udløb.

Projektet afgrænses fra at analysere kundernes adfærd efter overgang til et ordinært abonnement samt fra at inddrage længerevarende churn-forløb efter kampagneperiodens afslutning.

1.2 Datakilder

Projektet baseres på de udleveret datafiler fra Jyllands Posten, som indeholder tre datasæt:

behavior.csv: kundeadfærd i kampagneperioden

Cancellation.csv: churn-label + begrundelser

Subscription.csv: abonnements- og kampagneinfo

2 GDPR og etik

GDPR er en af de vigtigste ting at have styr på, når man arbejder med data. Det betyder, at Jyllands Posten skal have et lovligt grundlag for at bruge data, fx samtykke eller legitim interesse. Samtidig må de kun bruge de data, der er nødvendige, og de må ikke bruge dem til andre formål end det, de er indsamlet til. Derudover skal kunderne informeres tydeligt om, hvad deres data bliver brugt til, og data skal opbevares sikkert og slettes, når det ikke længere er relevant.

GDPR er også en vigtig retskilde i forhold til persondata og stiller krav til, hvordan data må bruges. For det første skal der være et behandlingsgrundlag (artikel 6), fx samtykke eller legitim interesse, som fx kan være analyse af churn. Hvis man bruger legitim interesse, skal man stadig tænke over kundens rettigheder og være gennemsigtig.

Der er også princippet om dataminimering, som betyder, at man kun må indsamle de data, man har brug for. Samtidig er der formålsbegrænsning, som betyder, at data kun må bruges til det formål, de er indsamlet til. Til sidst har kunderne rettigheder som indsigt og sletning, som virksomheden skal kunne håndtere.

3 AI og etik

Når det kommer til AI, er der også nogle etiske ting, man skal tænke over. Modellerne må fx ikke ende med at diskriminere bestemte kundegrupper eller træffe unfair beslutninger. Derfor er det vigtigt, at man kan forklare, hvorfor en kunde bliver vurderet til at churne. Samtidig bør AI bruges til at skabe værdi for kunden, fx gennem bedre og mere relevante tilbud, og ikke til at presse eller manipulere kunderne.

Når Jyllands Posten bruger AI til fx at forudsige churn, kan der opstå problemer med bias. Hvis modellen er trænet på gamle data, kan den komme til at videreføre skævheder. Fx hvis unge kunder har haft højere churn tidligere, kan modellen komme til at "straffe" dem, selvom årsagen måske ikke er retfærdig.

Derudover er transparens vigtig. Hvis en kunde får særlige tilbud eller bliver behandlet anderledes, bør man kunne forklare hvorfor. Hvis ikke, kan det skabe mistillid.

En anden ting er manipulation. Det kan være fint at give tilbud til kunder, der er ved at opsiges, men det må ikke blive på en måde, hvor kunder bliver presset uden at forstå det.

Samlet set handler det om, at AI skal bruges ansvarligt med fokus på fairness, gennemsigtighed og kundens oplevelse ikke kun profit.

3.1 AI Lovgivning

I forhold til AI lovgivning er der også nogle ting, Jyllands Posten skal have styr på. Selvom en churnmodel typisk bliver set som lav til middel risiko, er der stadig krav. De skal fx kunne forklare, hvordan modellen virker, og sikre at den ikke skaber problemer for kunderne.

Den nye EU AI Act stiller krav til, hvordan AI bliver udviklet og brugt. Selvom en churnmodel ikke er højrisiko, betyder det ikke, at man bare kan gøre hvad man vil. Jyllands Posten skal stadig have styr på dokumentation, fx hvordan modellen er trænet, hvilke data der er brugt, og hvad formålet er.

Derudover er gennemsigtighed vigtigt. Hvis AI bliver brugt til beslutninger, der påvirker kunder, skal man kunne forklare det. Ellers kan det skabe mistillid.

Hvis modellen senere bliver brugt mere direkte, fx til at ændre priser eller vilkår automatisk, kan den blive vurderet som højere risiko. Så kommer der endnu strengere krav.

Derfor er det vigtigt, at Jyllands-Posten løbende holder øje med deres AI og sikrer, at det overholder reglerne.

3.2 Cyber security

Til sidst er cybersikkerhed også en vigtig del. Jyllands-Posten skal beskytte deres data mod hacking og datalæk, fx ved brug af kryptering, adgangsstyring og overvågning af systemer. Det er både et krav og vigtigt for at bevare kundernes tillid.

Når man arbejder med mange abonnentdata, er det vigtigt at sikre, at uvedkommende ikke kan få adgang til dem. Det betyder fx, at data skal være krypteret både når de bliver gemt og når de bliver sendt. Samtidig skal der være styr på, hvem der har adgang, så det kun er relevante medarbejdere.

Derudover bør systemerne overvåges, så man kan opdage mistænkelig aktivitet hurtigt. Men cybersikkerhed handler ikke kun om teknik. Menneskelige fejl er ofte en stor risiko, så medarbejdere skal også have viden om fx phishing og hvordan man håndterer data korrekt.

Hvis der sker et databrud, skal Jyllands-Posten melde det inden for 72 timer og i nogle tilfælde også informere kunderne.

Samlet set er cybersikkerhed både et lovkrav og vigtigt for at opretholde kundernes tillid.

3.3 Delkonklusion

Konklusionen er derfor, at succes med en churn model ikke kun handler om, hvor god modellen er, men også hvordan den bliver brugt i praksis. Hvis Jyllands-Posten arbejder struktureret med GDPR, sørger for transparens i deres AI og har styr på cybersikkerhed, kan de både forbedre deres forretning og skabe bedre relationer til kunderne. Omvendt kan manglende fokus på de her ting føre til mistillid, problemer med lovgivning og i sidste ende tab af kunder. Derfor er det vigtigt, at datajura, etik og sikkerhed bliver tænkt ind fra starten og ikke først bagefter.

Samtidig kan man sige, at en ansvarlig brug af data og AI faktisk også kan være en fordel i konkurrencen. Kunder er blevet mere opmærksomme på deres data og rettigheder, og derfor betyder gennemsigtighed mere end før. Hvis Jyllands-Posten viser, at de håndterer data ordentligt og bruger AI på en fair måde, kan det være med til at skabe tillid.

Det betyder også, at det ikke kun handler om at tjene penge på kort sigt, men om at opbygge en løsning, der fungerer på lang sigt. Ved at kombinere gode analyser med ansvarlig brug af data kan Jyllands-Posten skabe værdi både for sig selv og for kunderne.

4 Databehandling

Der starts med at rense og klargøres data, så vi har et Datasæt for Jyllands Postens abonnementsdata der er klar til udviklingen af en Machine Learning model.

Først indlæser vi de nødvendige pakker og de udleveret datasæt, hvor vi arbejder med både abonnementsdata, afmeldingsdata og affærsdata. Herefter sørger vi for, at hver kunde kun optræder én gang ved at fjerne dubletter på `pseudo_id`. Det gør vi for at undgå fejl i analysen og sikre, at hver observation repræsenterer én kunde.

Derefter fletter vi datasættene sammen, så vi får alle relevante informationer samlet ét sted. Samtidig omdanner vi datoer til rigtige datoformater, så de kan bruges korrekt i beregninger. Det er vigtigt, fordi datoerne bruges til at definere churn.

Herefter laver vi de vigtigste variable. Først definerer vi `churn`, som viser, om kunden ikke fortsætter efter kampagnen. Hvis abonnementet stopper før eller ved kampagnens slutdato, er det `churn`. Hvis det fortsætter, er det ikke `churn`. Derefter laver vi `continued_subscription`, som bare er det modsatte af `churn`. Til sidst laver vi `early_churn`, som viser de kunder, der godt nok fortsætter, men opsiger kort tid efter. Her har vi sat grænsen til 90 dage efter kampagnen.

Efter det går vi i gang med datarensning og feature engineering. Vi fjerner `pseudo_id`, fordi det kun er en identifikator. Vi omdanner datoer og fjerner rækker med manglende værdier, så vi arbejder med valide data. Samtidig gør vi kategoriske variable som fx køn og afmeldingsårsag klar som faktorer. Manglende værdier bliver erstattet med ikke oplyst, så data stadig giver mening.

Derudover laver vi nye variable som alder og kundetid, som kan være vigtige for at forklare churn. Kundetid bliver også opdelt i grupper, så det er lettere at analysere. Til sidst fjerner vi de variable, vi ikke længere har brug for.

Til sidst tjekker vi datasættet og ser på fordelingen af churn og de andre variable. Til sidst gemmer vi det rensede datasæt i en RDS fil, så det er klar til videre analyse og modellering.

4.1 Machine learning modellen

I denne del af projektet har vi udviklet to Machine Learning-modeller, som tager udgangspunkt i Jyllands-Postens forretningsproblem omkring churn. Den første model har til formål at forudsige, hvilke kunder der cherner direkte efter kampagneperioden, og dermed også hvilke kunder der fortsætter. Den anden model bygger videre på den første og fokuserer kun på de kunder, der faktisk fortsatte efter kampagnen, for derefter at forudsige hvilke af dem der alligevel cherner efter kort tid. På den måde arbejder vi i to trin, som passer direkte til problemformuleringen i casen.

Vi starter med at indlæse det rensede datasæt `model_data.rds`, som allerede er klargjort gennem data cleaning og feature engineering. Derefter laver vi et nyt datasæt til model 1, hvor vi bevidst fjerner variable, som kan give leakage. Det gælder blandt andet `subscription_cancel_date`, `type`, `reason`, `expiration_date`, `first_campaign_day`, `last_campaign_day` og `continued_subscription`. Grunden til, at de bliver fjernet, er, at de enten direkte afslører udfaldet eller ligger for tæt op ad target-variablen. Hvis vi beholdt dem, ville modellen få information, den i praksis ikke burde have adgang til på prediction-tidspunktet, og så ville performance blive kunstigt høj. Vi beholder heller ikke `pseudo_id`, da det kun er en teknisk identifikator og ikke en reel forklarende variabel.

Selve target-variablen i model 1 er `churn`, som bliver lavet til en faktor med niveauerne 0 og 1. Herefter splitter vi datasættet i træningsdata og testdata med en 80/20 fordeling, hvor vi bruger stratificering på `churn`. Det gør vi for at sikre, at fordelingen mellem `churn` og ikke-`churn` ligner hinanden i både `train` og `test`. Efter det laver vi 5-fold cross validation på træningsdata, igen med stratificering. Formålet med det er at få en mere stabil og troværdig vurdering af modellernes performance under tuning.

I recipe-delen for model 1 forbereder vi data systematisk. Først fjerner vi `early_churn`, fordi den variabel er direkte relateret til fremtidig adfærd og derfor vil skabe leakage i `churn`-modellen. Derefter bruger vi `step_novel`, så modellen kan håndtere nye kategorier i testdata. Vi laver dummy-encoding af alle kategoriske variable med one-hot encoding, så modellerne kan arbejde numerisk med dem. Herefter fjerner vi zero variance variable, altså variable uden reel variation, og normaliserer de numeriske variable. Til sidst bruger vi `step_downsample(churn)`, fordi `churn`-klasserne sandsynligvis er ubalancerede. Downsampling gør, at majoritetsklassen reduceres i træningsdata, så modellerne ikke bare lærer at gætte den største klasse hele tiden.

Vi tester fem forskellige klassifikationsmodeller. Den første er et decision tree, hvor vi tuner `tree_depth`, `min_n` og `cost_complexity`. Den anden er logistisk regression med regularisering via `glmnet`, hvor vi tuner `penalty` og `mixture`. Den tredje er random forest med `ranger`, hvor vi tuner `mtry` og `min_n`, mens antal træer er sat til 500. Den fjerde er en SVM med radial basis function, hvor vi tuner `cost` og `rbf_sigma`. Den femte er XGBoost, hvor vi tuner blandt andet antal træer, trædybde, learning rate og `mtry`. På den måde sammenligner vi både simple, fortolkelige og mere avancerede modeller.

Alle modeller bliver samlet i et `workflow_set`, så de kan køres igennem den samme preprocessing pipeline. Derefter tuner vi dem med `tune_grid` på vores cross validation folds. Vi bruger flere evalueringsmål: `roc_auc`, `accuracy`, `f_meas`, `sens` og `spec`. Det er vigtigt, fordi `accuracy` alene ofte er misvisende ved ubalancerede klasser. Vi vælger den bedste model ud fra `f_meas`, da den balancerer `precision` og `recall` og derfor er særlig relevant i en churn-case, hvor det er vigtigt både at finde churn-kunder og undgå for mange falske alarmer.

Når den bedste model er fundet, henter vi de bedste hyperparametre og finaliserer workflowet. Derefter laver vi `last_fit` på det oprindelige split, så modellen bliver trænet på hele træningsdatasættet og evalueret på testdatasættet. Til sidst indsamler vi `metrics`, laver en confusion matrix og en ROC-curve. Confusion matrixen giver os et konkret billede af, hvor mange kunder modellen klassificerer korrekt og forkert, mens ROC-kurven viser modellens evne til at skelne mellem `churn` og ikke-`churn` på tværs af thresholds.

Model 2 er bygget op på næsten samme måde, men her fokuserer vi kun på kunder, som allerede har `continued` efter kampagnen, altså hvor `churn == 0`. Formålet med denne model er at identificere `early churn`, altså kunder som først fortsætter, men alligevel falder fra kort tid efter. Det gør modellen mere forretningsmæssigt relevant, fordi Jyllands-Posten så ikke kun kan forstå det første frafald, men også den gruppe af kunder, der ser ud til at være vundet, men som stadig er ustabile.

I model 2 laver vi derfor et nyt datasæt `early_data`, hvor target-variablen er `early_churn`. Vi splitter igen i `train` og `test` med stratificering og laver 5-fold cross validation. I recipe'en fjerner vi `churn`, fordi det er parent target og ikke skal bruges som predictor i `early churn`-modellen. Ellers følger preprocessing stort set samme logik som i model 1: håndtering af nye kategorier, dummy-encoding, fjernelse af zero variance og normalisering. Her har vi valgt ikke at bruge downsampling i den nuværende version, men det er kommenteret i koden, så det kan tilføjes senere, hvis klasseubalancen viser sig at være et større problem.

Vi genbruger de samme fem modeltyper, så vi kan sammenligne resultaterne på tværs af de to opgaver. Også her vælger vi den bedste model på baggrund af `f_meas`, finaliserer workflowet og evaluerer på testdatasættet. Resultatet bliver igen vurderet med `metrics` og confusion matrix.

Samlet set er styrken ved denne ML-opsætning, at den er systematisk, sammenlignende og forholdsvis robust. Vi arbejder ikke kun med én model, men tester flere forskellige algoritmer. Vi håndterer leakage bevidst, laver

preprocessing gennem recipes, bruger cross validation og hyperparameter tuning, og evaluerer modellerne med flere relevante metrics. Samtidig er modellen bygget direkte op omkring Jyllands-Postens forretningsproblem, fordi vi både forudsiger det første churn efter kampagnen og det efterfølgende early churn blandt de kunder, der i første omgang bliver fastholdt. Det gør løsningen mere brugbar i praksis, fordi resultaterne både kan bruges til at identificere risikokunder og til at planlægge mere målrettede retention-indsatser.

4.2 Kundedadfærdsanalyse

I dette afsnit kigger vi på kundedadfærd ud fra vores data i behavior.csv. Formålet er at forstå, hvordan brugerne opfører sig på siden, og om vi kan finde nogle mønstre, som kan bruges videre i forhold til churn.

Først laver vi data om til kundeniveau, så hver kunde kun har én række. Her laver vi nogle variable, som beskriver adfærden, fx hvor mange besøg en bruger har (n_visits), hvor langt de scroller (avg_scroll), og om de bruger mobil eller computer. Vi ser også på, hvor meget de er inde på paywall indhold, hvor de kommer fra (search eller intern), og hvor mange forskellige sider de besøger. Det giver et ret godt billede af, hvor aktive og engagerede kunderne er.

Derefter standardiserer vi data og laver en PCA. Det gør vi for at gøre data mere simpelt og samle informationen i færre variable, som stadig forklarer det meste. På den måde bliver det nemmere at arbejde videre med.

Efter det laver vi klyngeanalyse, hvor vi prøver at opdele kunderne i 3 grupper baseret på deres adfærd. Vi bruger både hierarkisk clustering og k-means for at være mere sikre på vores opdeling. Når vi har klyngerne, kigger vi på gennemsnit for hver gruppe, så vi kan forstå forskellene og lave en profil af hver kundetype.

Til sidst tester vi, om der faktisk er forskelle mellem klyngerne med ANOVA, og vi laver også nogle plots for at visualisere det.

Samlet set hjælper analysen os med at forstå kundedadfærd bedre og gør det muligt at opdele kunderne i forskellige typer, som vi senere kan bruge i vores churn analyse.

4.3 Evaluering af kode

I denne del af projektet evaluerer vi vores Machine Learning-modeller for at vurdere, hvor godt de faktisk kan forudsige churn. Vi bruger flere forskellige metrics som accuracy, ROC-AUC, sensitivity og F1-score. Grunden til, at vi ikke kun bruger én metric, er, at datasættet er skævt fordelt, hvor der typisk er flere kunder, der ikke cherner, end kunder der cherner. Derfor kan accuracy alene give et misvisende billede af modellens performance.

Vi har især fokus på F1-score, fordi den balancerer mellem precision og recall. I en forretningskontekst er det vigtigt både at identificere de kunder, der faktisk cherner (undgå false negatives), og samtidig undgå at ramme for mange kunder forkert med retention-indsatser (false positives). Hvis modellen fx overser churn-kunder, mister virksomheden potentielle indtægter, og hvis den rammer forkert, kan det føre til unødvendige marketingomkostninger.

Derudover bruger vi ROC-AUC til at vurdere modellens evne til at skelne mellem churn og ikke-churn på tværs af forskellige thresholds. Sensitivity er også relevant, da den viser, hvor stor en andel af churn-kunderne modellen faktisk fanger.

Resultaterne viser, at ensemble-modeller som random forest og XGBoost generelt performer bedst. Det giver god mening, da de er bedre til at håndtere komplekse sammenhænge og mange variable i data. Samlet set giver evalueringen et solidt grundlag for at vælge den mest brugbare model i forhold til Jyllands-Postens forretningsbehov.

5 Datavisualisering

I dette projekt spiller datavisualisering en central rolle, fordi det er med til at omsætte vores analyser og modeller til noget, Jyllands-Posten rent faktisk kan bruge i praksis. Når man arbejder med churn, kundedadfærd

og segmentering, kan resultaterne hurtigt blive tekniske og svære at læse direkte ud fra tabeller eller modeloutput. Derfor bruger vi både Shiny og Power BI til at visualisere data, så resultaterne bliver mere overskuelige, interaktive og lettere at formidle til virksomheden.

I vores kode arbejder vi med flere forskellige typer visualiseringer alt efter, hvad vi gerne vil undersøge. I adfærdsanalysen bruger vi fx histogrammer til at undersøge fordelingen af variable som antal besøg, scroll depth og antal unikke sider. Formålet med histogrammerne er at få et overblik over, hvordan data fordeler sig, og om der fx er skævheder, outliers eller meget store forskelle mellem kunderne. Det er vigtigt i en tidlig fase, fordi det giver en bedre forståelse af datasættet og hjælper os med at vurdere, hvordan data skal behandles videre.

Derudover bruger vi boxplots til at sammenligne forskelle mellem de kundeklynger, vi finder i vores segmentering. I koden laver vi fx boxplots for `n_visits`, `avg_scroll`, `share_mobile` og `share_search` fordelt på de klynger, der er fundet med k-means. De diagrammer er gode, fordi de ikke bare viser gennemsnit, men også spredning, median og eventuelle outliers. Det gør det lettere at se, om en bestemt klynge fx har højere aktivitet, større mobilandel eller mere trafik fra search end de andre grupper. På den måde bliver det nemmere at lave profiler af kundetyperne.

Vi bruger også PCA som en del af visualiseringen. Her laver vi først en standardisering af data og derefter en Principal Component Analysis, så vi kan reducere antallet af variable og samle informationen i færre komponenter. Til dette bruger vi blandt andet screeplots og biplots. Screeplottet hjælper os med at vurdere, hvor mange komponenter der giver mening at arbejde videre med, fordi det viser, hvor meget variation hver komponent forklarer. Biplottet gør det muligt at se relationen mellem observationer og variable i et reduceret rum, hvilket giver en visuel forståelse af, hvordan kundeadfærden hænger sammen. Det er især nyttigt før klyngeanalysen, fordi vi kan få en idé om, om der faktisk er naturlige grupper i data.

I forhold til selve segmenteringen bruger vi også visualisering i forbindelse med hierarkisk clustering. Her plotter vi dendrogrammet fra `hclust`, så vi kan se, hvordan kunderne grupperer sig, og hvor det giver mening at skære træet for at få et bestemt antal klynger. Det er med til at underbygge, hvorfor vi vælger fx 3 klynger i vores videre analyse.

Når vi går fra analyse til formidling, bruger vi Shiny til at gøre visualiseringerne interaktive. Det betyder, at brugeren selv kan vælge variable, filtre og visninger. Det gør løsningen mere fleksibel, fordi Jyllands-Posten ikke bare ser et statisk billede, men selv kan udforske data. I Shiny kan man fx forestille sig, at brugeren vælger en bestemt kundeklynge og derefter ser udviklingen i churn, fordeling af adfærd eller forskelle i abonnementsmønstre. Det giver meget mere værdi end en fast rapport, fordi virksomheden selv kan stille spørgsmål til data og få svar med det samme.

Power BI bruger vi mere til dashboards og ledelsesoverblik. Hvor Shiny er god til interaktiv analyse, er Power BI stærk til at samle KPI'er og vigtige resultater i et mere forretningsorienteret format. Her kan vi fx vise churn-rate, andel continued subscription, early churn, segmentfordelinger og udviklinger over tid i et dashboard. Vi kan også bruge slicers og filtre, så Jyllands-Posten kan se resultater for bestemte kundegrupper eller perioder. Diagrammer i Power BI kan fx være søjlediagrammer til at sammenligne churn mellem segmenter, linjediagrammer til udvikling over tid og kort eller tabeller til at vise mere detaljeret information.

Det vigtige er derfor ikke kun, at vi visualiserer data, men også hvordan vi gør det. Histogrammer hjælper os med at forstå fordelingen i data. Boxplots hjælper os med at sammenligne grupper. Screeplots, biplots og dendrogrammer hjælper os med at forstå vores PCA og klyngeanalyse. Shiny giver mulighed for interaktiv udforskning, mens Power BI giver et hurtigt og professionelt overblik over de vigtigste resultater.

Samlet set er datavisualisering en meget vigtig del af projektet, fordi det binder analyse og forretning sammen. Det gør vores resultater mere forståelige og mere anvendelige for Jyllands-Posten. Ved at kombinere kodebaserede visualiseringer i R med interaktive løsninger i Shiny og dashboards i Power BI får vi både en analytisk og en praktisk måde at arbejde med resultaterne på. Det gør, at vores projekt ikke kun viser, hvad modellen kan, men også hvordan indsigterne kan bruges i praksis.

5.1 Evaluerings af forretningsmæssige fordele

I en forretningsmæssig sammenhæng kan denne analyse direkte være med til at adressere Jyllands Postens problem med høj churn efter kampagneperioder. Ved at bruge Machine Learning til at forudsige churn, får

virksomheden mulighed for at identificere hvilke kunder, der er i risiko for at opsige deres abonnement allerede inden det sker. Det gør det muligt at arbejde proaktivt i stedet for reaktivt.

Samtidig giver analysen indsigt i, hvilke faktorer der driver churn. Ved at kombinere abonnementsdata og adfærdsdata kan Jyllands Posten forstå, hvilke typer kunder der cherner, og hvad der kendetegner dem. Fx kan det være lav aktivitet, få besøg eller manglende engagement med indhold. Denne viden gør det muligt at målrette indsatsen, så man ikke bruger ressourcer på alle kunder, men fokuserer på dem med størst risiko.

Kundesegmenteringen spiller også en vigtig rolle, da den opdeler kunderne i forskellige grupper baseret på deres adfærd. Det gør det muligt at tilpasse både kommunikation, tilbud og onboarding. Fx kan nye kunder med lav aktivitet få ekstra hjælp til at komme i gang, mens mere aktive kunder kan få mere relevant indhold eller premium tilbud. På den måde kan Jyllands-Posten skabe mere værdi for den enkelte kunde og dermed øge sandsynligheden for fastholdelse.

Derudover gør visualiseringerne i Shiny og Power BI det muligt at omsætte analysen til et konkret værktøj for forretningen. I stedet for kun at have tekniske modelresultater, får Jyllands Posten et dashboard, hvor de kan følge churn, identificere risikokunder og se mønstre i data. Et gør løsningen anvendelig for ikke-tekniske brugere og understøtter bedre beslutningstagning. Ud fra det kan det konkluderes, at Jyllands Posten kan arbejde mere datadrevet med både fastholdelse og fremtidige kampagner. I stedet for at fokusere på volumen af nye abonnenter, kan de i højere grad fokusere på kvaliteten af kunderne og deres langsigtede værdi. Dette kan være med til at reducere churn, øge kundeloyaliteten og skabe en mere bæredygtig abonnementsforretning.

6 Kode

6.1 Datarensning

```
pacman::p_load(tidyverse, lubridate, forcats, readr)

# Indlæs data
cancellation <- read.csv("data/cancellation.csv")
subscription <- read.csv("data/subscription_v2.csv", sep = ";")
# behavior <- read.csv("data/behavior.csv")

# Behold kun 1 ID per row (seneste dato pr. tabel)
# cancellation → brug expiration_date
cancellation <- cancellation %>%
  mutate(expiration_date = ymd(expiration_date)) %>%
  group_by(pseudo_id) %>%
  slice_max(expiration_date, n = 1, with_ties = FALSE) %>%
  ungroup()

# subscription → brug subscription_cancel_date
subscription <- subscription %>%
  mutate(subscription_cancel_date = dmy(subscription_cancel_date)) %>%
  group_by(pseudo_id) %>%
  slice_max(subscription_cancel_date, n = 1, with_ties = FALSE) %>%
  ungroup()

# Flet data + churn-definition
merged_data <- subscription %>%
  left_join(cancellation, by = "pseudo_id") %>%

# Parse dates
mutate(
```

```

    subscription_cancel_date = as.Date(subscription_cancel_date),
    first_campaign_day = dmy(first_campaign_day),
    last_campaign_day = dmy(last_campaign_day),
    expiration_date = as.Date(expiration_date)
  ) %>%

# Churn
mutate(
  churn = case_when(
    is.na(expiration_date) ~ 0,
    expiration_date <= last_campaign_day ~ 1,
    expiration_date > last_campaign_day ~ 0
  )
) %>%

# Continued subscription
mutate(
  continued_subscription = 1 - churn
) %>%

# Early churn
mutate(
  early_churn = case_when(
    continued_subscription == 1 & !is.na(expiration_date) &
      expiration_date <= last_campaign_day + 90 ~ 1,
    continued_subscription == 1 ~ 0,
    continued_subscription == 0 ~ 0
  )
)

# Churn distribution
merged_data %>%
  count(churn) %>%
  mutate(prop = n / sum(n))

```

A tibble: 2 x 3

	churn	n	prop
	<dbl>	<int>	<dbl>
1	0	742	0.580
2	1	537	0.420

```

# Feature engineering / rensning
model_data <- merged_data %>%
  mutate(
    order_date = as.Date(ymd_hms(order_date)),
    birthdate = as.Date(parse_date_time(birthdate, orders = c("dmy", "ymd", "mdy"))),
    usr_created = dmy(usr_created)
  ) %>%

# Fjern NA
filter(!is.na(birthdate), !is.na(usr_created), !is.na(order_date)) %>%

# Kategorisk rensning
mutate(
  churn = factor(churn),

```



```

    type = factor(type),
    reason = factor(reason),
    koen = factor(koen),
    koen = fct_recode(koen, "Ikke oplyst" = ""),
    type = fct_na_value_to_level(type, level = "Ingen afmelding"),
    reason = fct_na_value_to_level(reason, level = "Ingen afmelding"),
    expiration_date = replace_na(expiration_date, as.Date("3000-01-01"))
  ) %>%

# Samtykker
mutate(
  permission_given_order = factor(if_else(permission_given_order == "true", 1, 0)),
  permission_given_today = factor(if_else(permission_given_today == "true", 1, 0))
) %>%

# Afledte variable
mutate(
  age = as.numeric(Sys.Date() - birthdate) / 365,
  kundetid_dage = as.numeric(order_date - usr_created)
) %>%

# Ekstra sikkerhed
filter(!is.na(kundetid_dage)) %>%

# Kundetid grupper
mutate(
  kundetid_gruppe = case_when(
    kundetid_dage <= 7 ~ "0-7 dage",
    kundetid_dage <= 30 ~ "8-30 dage",
    kundetid_dage <= 180 ~ "1-6 måneder",
    TRUE ~ "6+ måneder"
  ) %>% factor()
)

# Fjern ubrugte kolonner
model_data <- model_data %>%
  select(
    -kundetid_dage,
    -order_date,
    -birthdate,
    -usr_created
  )

# Tjek
glimpse(model_data)

```

Rows: 1,275

Columns: 22

```

$ pseudo_id      <chr> "003a92354fb88e0e3e411bc290291b6a", "004b2a16~
$ account_active_days <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ subscription_cancel_date <date> 2025-05-11, 2025-04-29, 2025-03-18, 2025-05-~
$ koen           <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ order_trackertag <chr> "utm_campaign=Marts25&utm_content=b&utm_mediu~
$ permission_given_order <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, ~

```

```

$ previous_subscriptions <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1, ~
$ previous_trials <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0, ~
$ first_campaign_day <date> 2025-03-05, 2025-03-05, 2025-03-18, 2025-03-~
$ last_campaign_day <date> 2025-05-04, 2025-05-04, 2025-05-17, 2025-05-~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ type <fct> Andet, Andet, Andet, Produkt, Tid, Andet, Tid~
$ reason <fct> kunden ønsker ikke at oplyse, Ingen årsag - i~
$ expiration_date <date> 2025-06-04, 2025-05-04, 2025-05-17, 2025-06-~
$ churn <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ continued_subscription <dbl> 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, ~
$ early_churn <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age <dbl> 76.33699, 69.78630, 39.75342, 45.69041, 74.29~
$ kundetid_gruppe <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~

```

```
summary(model_data$age)
```

```

      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
0.3041  50.9699  61.4959  60.2016  71.5397 143.0712

```

```
colSums(is.na(model_data))
```

```

      pseudo_id      account_active_days subscription_cancel_date
           0                0                0
      koen      order_trackertag      permission_given_order
           0                0                0
permission_given_today previous_subscriptions      previous_campaigns
           0                0                0
      previous_trials      first_campaign_day      last_campaign_day
           0                0                0
newsletters_before_order newsletters_after_order      type
           0                0                0
      reason      expiration_date      churn
           0                0                0
continued_subscription      early_churn      age
           0                0                0
      kundetid_gruppe
           0

```

```

# Churn distribution
model_data %>%
  count(churn) %>%
  mutate(prop = n / sum(n))

```

```

# A tibble: 2 x 3
  churn     n prop
  <fct> <int> <dbl>
1 0       740 0.580
2 1       535 0.420

```

```
model_data %>%
  count(continued_subscription) %>%
  mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
  continued_subscription     n prop
      <dbl> <int> <dbl>
1             0     535 0.420
2             1     740 0.580
```

```
model_data %>%
  count(early_churn) %>%
  mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
  early_churn     n prop
      <dbl> <int> <dbl>
1             0  1037 0.813
2             1   238 0.187
```

```
glimpse(model_data)
```

```
Rows: 1,275
Columns: 22
$ pseudo_id      <chr> "003a92354fb88e0e3e411bc290291b6a", "004b2a16~
$ account_active_days <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ subscription_cancel_date <date> 2025-05-11, 2025-04-29, 2025-03-18, 2025-05-~
$ koen           <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ order_trackertag <chr> "utm_campaign=Marts25&utm_content=b&utm_mediu~
$ permission_given_order <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, ~
$ previous_subscriptions <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ first_campaign_day <date> 2025-03-05, 2025-03-05, 2025-03-18, 2025-03-~
$ last_campaign_day <date> 2025-05-04, 2025-05-04, 2025-05-17, 2025-05-~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ type           <fct> Andet, Andet, Andet, Produkt, Tid, Andet, Tid~
$ reason         <fct> kunden ønsker ikke at oplyse, Ingen årsag - i~
$ expiration_date <date> 2025-06-04, 2025-05-04, 2025-05-17, 2025-06-~
$ churn          <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ continued_subscription <dbl> 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, ~
$ early_churn      <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age            <dbl> 76.33699, 69.78630, 39.75342, 45.69041, 74.29~
$ kundetid_gruppe <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~
```

```
view(model_data)
```

```
# Eksport
write_csv(model_data, "data/model_data.csv")
saveRDS(model_data, "data/model_data.rds")
```

6.2 Klyngeanalyse

```
# -----
# 1. Load packages
# -----
pacman::p_load(tidyverse, DataExplorer, ggpubr)

# -----
# 2. Load data
# -----
behavior <- read_csv("data/behavior.csv")
model_data <- readRDS("data/model_data.rds")

glimpse(behavior)

Rows: 265,456
Columns: 10
$ pseudo_id      <chr> "fcfd45344b1647471f5ec6194c220c3e", "743a9d6d44dfa092~
$ dt             <chr> "2025-03-13", "2025-03-13", "2025-03-13", "2025-04-17~
$ collector_tstamp <chr> "2025-03-13 03:05:31.679000", "2025-03-13 01:24:24.26~
$ dvce_type      <chr> "Mobile", "Computer", "Computer", "Mobile", "Computer~
$ os_family      <chr> "Android", "Mac OS X", "Mac OS X", "iOS", "Mac OS X",~
$ page_url_clean  <chr> "https://jyllands-posten.dk/", "https://jyllands-post~
$ refr_medium    <chr> "search", "", "internal", "internal", "", "", "intern~
$ page_restricted <chr> "no", "no", "no", "no", "no", "no", "yes", "yes", "no~
$ scroll_depth    <dbl> 0.00, 0.00, 1.00, 1.00, 0.00, 0.00, 0.00, 1.00, 0.00,~
$ webpage_id     <chr> "d12e9d666f4f9a3a62573d8d82c8a7d3", "5bb6b8db23b4ab77~

# -----
# 3. Create behavioral customer-level features
# -----
customer_behavior <- behavior %>%
  group_by(pseudo_id) %>%
  summarise(
    n_visits = n(),
    avg_scroll = mean(scroll_depth, na.rm = TRUE),
    share_mobile = mean(dvce_type == "Mobile"),
    share_desktop = mean(dvce_type == "Computer"),
    share_restricted = mean(page_restricted == "yes"),
    share_search = mean(refr_medium == "search"),
    share_internal = mean(refr_medium == "internal"),
    n_unique_pages = n_distinct(webpage_id)
  )

# -----
# 4. Prepare subscription features
# -----
subscription_features <- model_data %>%
  select(
    pseudo_id,
    account_active_days,
    previous_subscriptions,
    previous_campaigns,
    previous_trials,
    newsletters_before_order,
```

```

    newsletters_after_order,
    churn
  )
# -----
# 5. MERGE DATA
# -----
customer_df <- subscription_features %>%
  left_join(customer_behavior, by = "pseudo_id")

# Indsæt 0 i stedet for NA for de 70 "Spørgelses-brugere"
customer_df <- customer_df %>%
  mutate(across(
    c(n_visits, n_unique_pages, avg_scroll, starts_with("share_")),
    ~replace_na(.x, 0)
  ))

# sanity check - should now be 1,275
nrow(customer_df)

```

[1] 1275

```

# check
colSums(is.na(customer_df))

```

pseudo_id	account_active_days	previous_subscriptions
0	0	0
previous_campaigns	previous_trials	newsletters_before_order
0	0	0
newsletters_after_order	churn	n_visits
0	0	0
avg_scroll	share_mobile	share_desktop
0	0	0
share_restricted	share_search	share_internal
0	0	0
n_unique_pages		
0		

```

# -----
# 6. Remove ID + handle missing values
# -----
cluster_input <- customer_df %>%
  select(-pseudo_id, -churn)

# check
glimpse(cluster_input)

```

Rows: 1,275

Columns: 14

```

$ account_active_days    <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ previous_subscriptions <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns     <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials        <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~

```

```

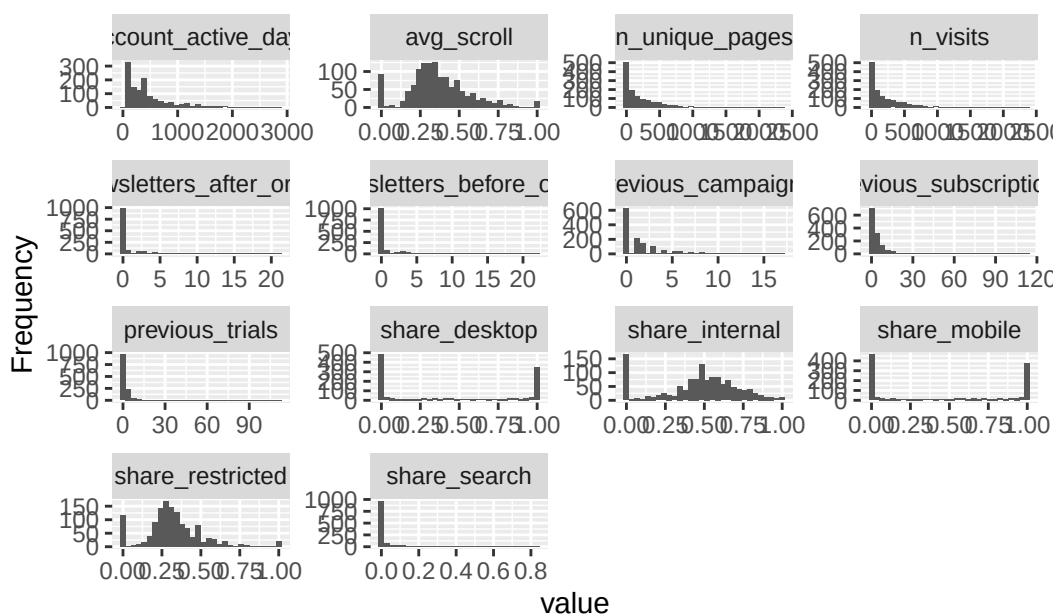
$ newsletters_after_order <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ n_visits <int> 517, 603, 15, 8, 46, 253, 201, 15, 80, 772, 2~
$ avg_scroll <dbl> 0.3471954, 0.4179104, 0.3666667, 0.6250000, 0~
$ share_mobile <dbl> 0.47969052, 0.83084577, 1.00000000, 1.0000000~
$ share_desktop <dbl> 0.5203095, 0.0000000, 0.0000000, 0.0000000, 1~
$ share_restricted <dbl> 0.2978723, 0.3200663, 0.3333333, 0.6250000, 0~
$ share_search <dbl> 0.015473888, 0.439469320, 0.000000000, 0.0000~
$ share_internal <dbl> 0.52417795, 0.53565506, 0.46666667, 0.0000000~
$ n_unique_pages <int> 516, 603, 15, 8, 46, 253, 201, 15, 80, 771, 2~

```

```

# -----
# 7. Explore distributions
# -----
plot_histogram(cluster_input)

```



```

# -----
# 8. Scale variables
# -----
customer_scaled <- scale(cluster_input)

# -----
# 9. PCA
# -----
pca <- prcomp(customer_scaled, scale = TRUE)

summary(pca)

```

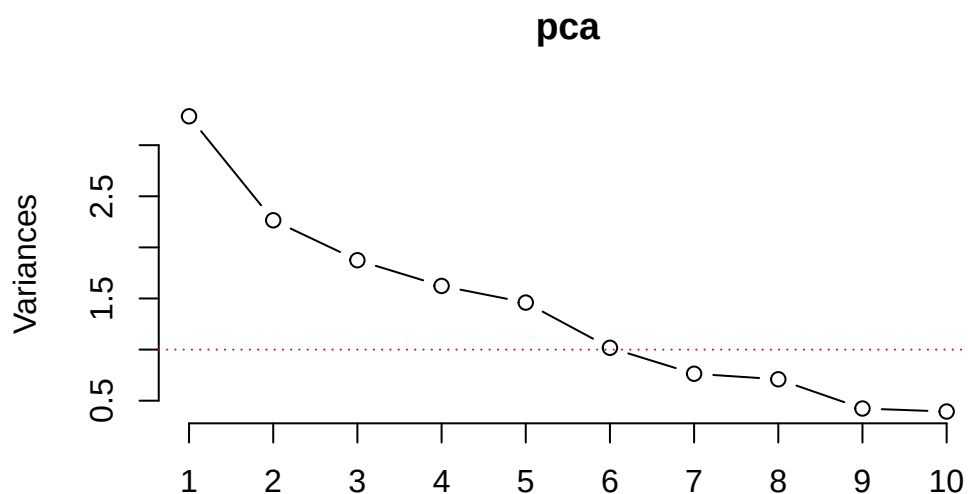
Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	1.8118	1.5050	1.3691	1.2739	1.2084	1.00873	0.87394
Proportion of Variance	0.2345	0.1618	0.1339	0.1159	0.1043	0.07268	0.05455
Cumulative Proportion	0.2345	0.3963	0.5302	0.6461	0.7504	0.82304	0.87760

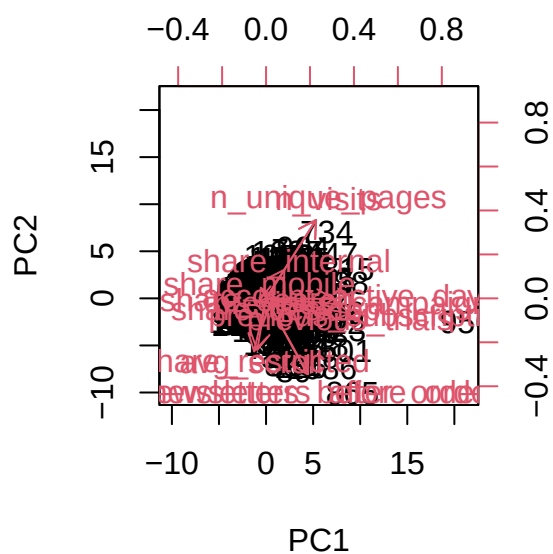
	PC8	PC9	PC10	PC11	PC12	PC13	PC14
--	-----	-----	------	------	------	------	------

Standard deviation	0.84229	0.65074	0.62764	0.34835	0.22119	0.12194	0.04013
Proportion of Variance	0.05068	0.03025	0.02814	0.00867	0.00349	0.00106	0.00012
Cumulative Proportion	0.92827	0.95852	0.98666	0.99533	0.99882	0.99988	1.00000

```
screeplot(pca, type = "line")
abline(h = 1, col = "red", lty = 3)
```



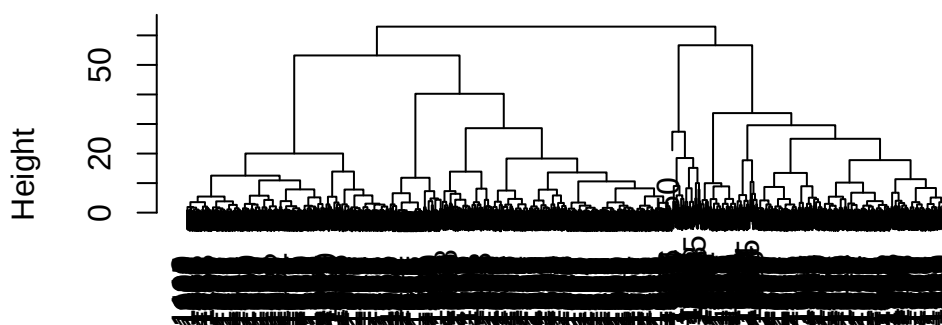
```
biplot(pca, scale = 0)
```



```
# -----
# 10. Hierarchical clustering
# -----
# hc_complete <- hclust(dist(pca$x[,1:4]), method = "complete")
# plot(hc_complete)
#
# customer_df <- customer_df %>%
#   filter(complete.cases(select(., -pseudo_id, -churn)))
#
# customer_df$cluster_hc <- cutree(hc_complete, 5)
#
# table(customer_df$cluster_hc)

hc_ward <- hclust(dist(pca$x[,1:4]), method = "ward.D2")
plot(hc_ward)
```

Cluster Dendrogram



```
dist(pca$x[, 1:4])
hclust (*, "ward.D2")
```

```
customer_df <- customer_df %>%
  filter(complete.cases(select(., -pseudo_id, -churn)))

customer_df$cluster_hc <- cutree(hc_ward, 5)

table(customer_df$cluster_hc)
```

```
  1   2   3   4   5
411 387 341 48  88
```

```
# -----
# 11. K-means clustering
# -----
set.seed(1)
```



```
km <- kmeans(pca$x[,1:4], centers = 5, nstart = 20)

customer_df$cluster_km <- km$cluster

table(customer_df$cluster_km)
```

```
  1    2    3    4    5
249 430 133 363 100
```

```
# compare methods
table(customer_df$cluster_km, customer_df$cluster_hc)
```

```
      1    2    3    4    5
1 245    4    0    0    0
2  81   18 331    0    0
3  48   34    4  47    0
4  36  321    5    1    0
5    1   10    1    0  88
```

```
# -----
# 12. Cluster profiling
# -----
customer_df$cluster_km <- as.factor(customer_df$cluster_km)

cluster_profile <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(across(where(is.numeric), mean, na.rm = TRUE))
```

Warning: There was 1 warning in `summarise()`.

i In argument: `across(where(is.numeric), mean, na.rm = TRUE)`.

i In group 1: `cluster\_km = 1`.

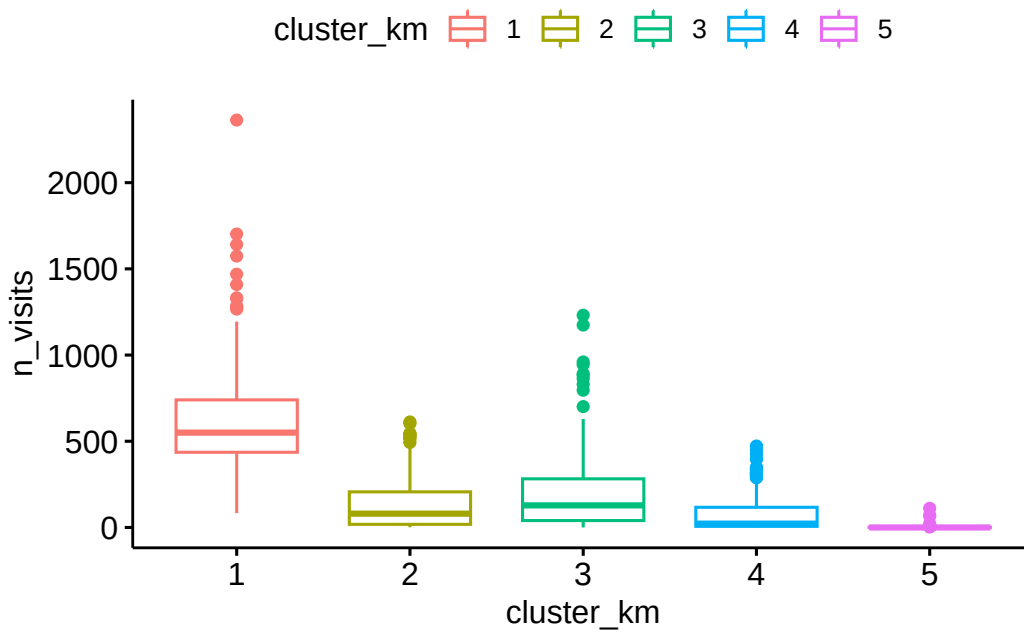
Caused by warning:

! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
Supply arguments directly to `fns` through an anonymous function instead.

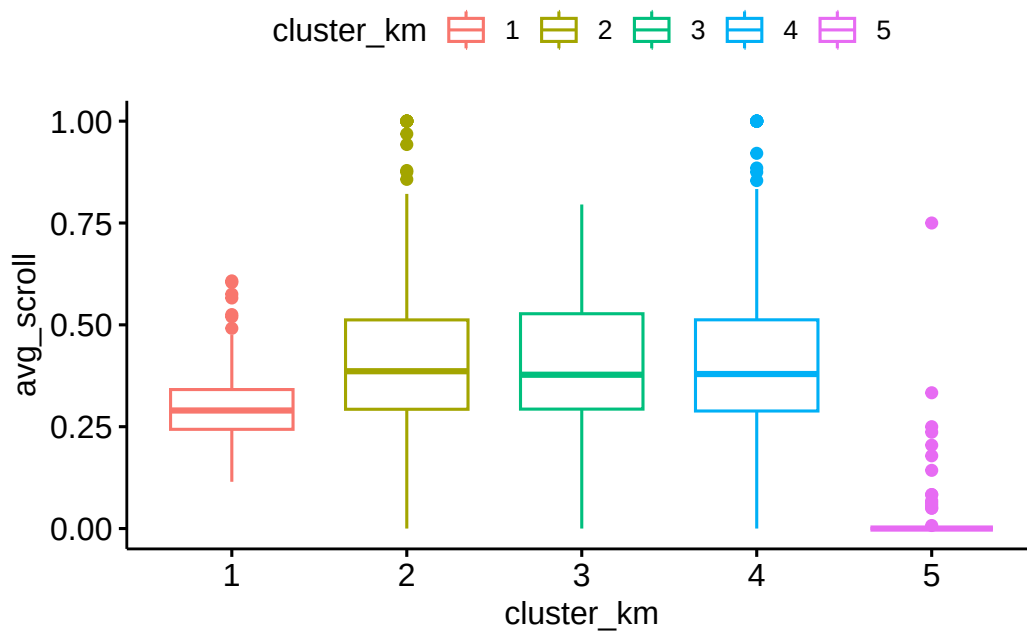
```
# Previously
across(a:b, mean, na.rm = TRUE)

# Now
across(a:b, \(x) mean(x, na.rm = TRUE))
```

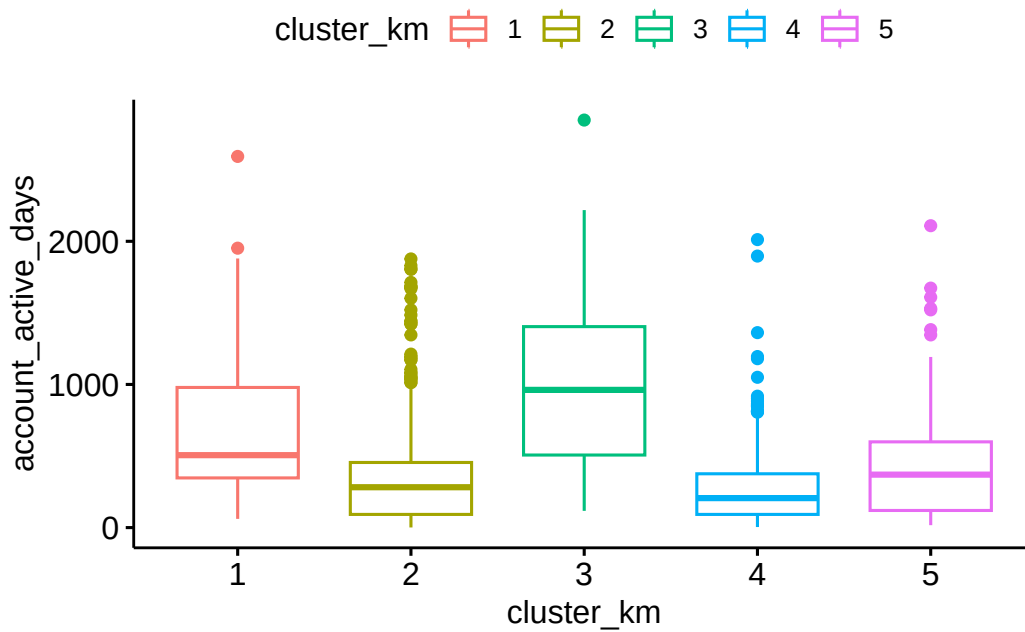
```
# -----
# 13. Visualisation
# -----
ggboxplot(customer_df, x = "cluster_km", y = "n_visits",
           color = "cluster_km")
```



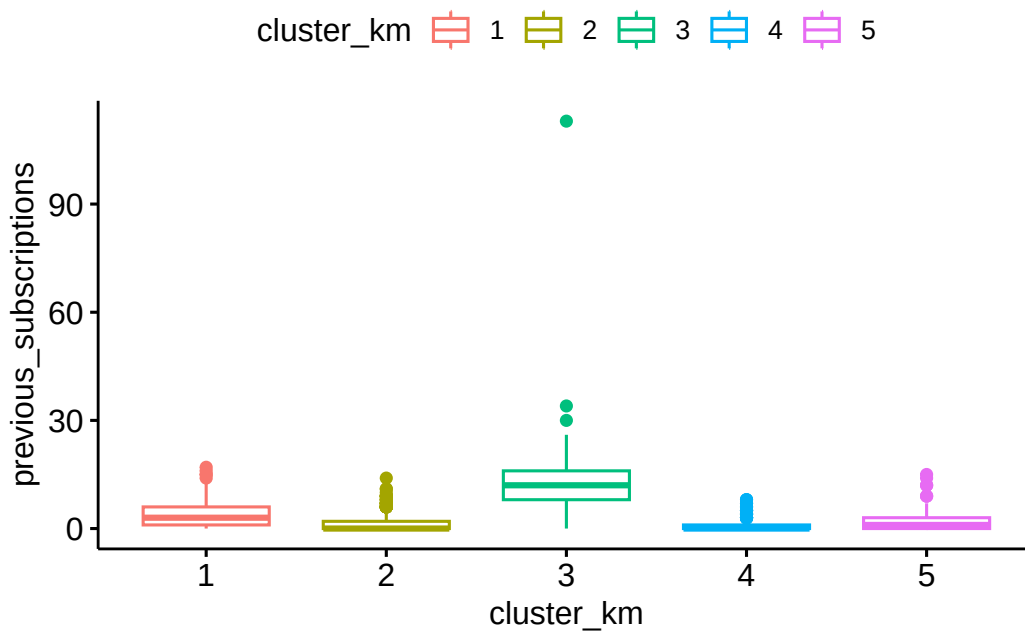
```
ggboxplot(customer_df, x = "cluster_km", y = "avg_scroll",
           color = "cluster_km")
```



```
ggboxplot(customer_df, x = "cluster_km", y = "account_active_days",
           color = "cluster_km")
```



```
ggboxplot(customer_df, x = "cluster_km", y = "previous_subscriptions",
           color = "cluster_km")
```



```
# -----
# 14. ANOVA tests
# -----
summary(aov(n_visits ~ cluster_km, data = customer_df))
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
cluster_km	4	53943667	13485917	407.3	<2e-16 ***

```
Residuals    1270 42052029    33112
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(avg_scroll ~ cluster_km, data = customer_df))
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km     4   15.52    3.881    145 <2e-16 ***
Residuals    1270   34.00    0.027
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(account_active_days ~ cluster_km, data = customer_df))
```

```
              Df    Sum Sq Mean Sq F value Pr(>F)
cluster_km     4 64845568 16211392   109.5 <2e-16 ***
Residuals    1270 187992176   148025
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_subscriptions ~ cluster_km, data = customer_df))
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km     4   15919    3980   208.9 <2e-16 ***
Residuals    1270   24198     19
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_campaigns ~ cluster_km, data = customer_df))
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km     4    2823    705.6    191 <2e-16 ***
Residuals    1270    4693     3.7
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_trials ~ cluster_km, data = customer_df))
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km     4    4996   1249.1    97.3 <2e-16 ***
Residuals    1270   16303    12.8
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(newsletters_before_order ~ cluster_km, data = customer_df))
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km     4    1814    453.5   149.5 <2e-16 ***
Residuals    1270    3852     3.0
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(newsletters_after_order ~ cluster_km, data = customer_df))
```

```

              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   1915   478.6   140.9 <2e-16 ***
Residuals   1270   4316     3.4
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

# -----
# 15. CHURN ANALYSIS (IMPORTANT)
# -----
churn_by_cluster <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(
    churn_rate = mean(churn == 1, na.rm = TRUE),
    n = n()
  )

print(churn_by_cluster)

```

```

# A tibble: 5 x 3
  cluster_km churn_rate     n
  <fct>      <dbl> <int>
1 1          0.470   249
2 2          0.419   430
3 3          0.489   133
4 4          0.369   363
5 5          0.39    100

```

```

# -----
# PRINT CLUSTER MEANS (COORDINATES)
# -----
cluster_summary_long <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE))) %>%
  pivot_longer(cols = -cluster_km, names_to = "variable", values_to = "mean_value") %>%
  pivot_wider(names_from = cluster_km, names_prefix = "Cluster_", values_from = mean_value)

# This will print the full table in your console
print(cluster_summary_long, n = 50)

```

```

# A tibble: 15 x 6
  variable Cluster_1 Cluster_2 Cluster_3 Cluster_4 Cluster_5
  <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
1 account_active_days 666.    373.    998.    273.    455.
2 previous_subscriptions 4.02    1.53   13.0    0.989    2.27
3 previous_campaigns 2.33    0.888   5.55    0.565    1.27
4 previous_trials 1.37    0.574   7.10    0.402    0.91
5 newsletters_before_order 0.309   0.302   4.23    0.361    0.42
6 newsletters_after_order 0.345   0.349   4.39    0.435    0.45
7 n_visits 613.    128.    215.    77.4     3.7
8 avg_scroll 0.300   0.420   0.411   0.426    0.0256
9 share_mobile 0.679   0.0643  0.425   0.908    0.162

```

10	share_desktop	0.279	0.909	0.530	0.0641	0.0927
11	share_restricted	0.277	0.346	0.366	0.393	0.0254
12	share_search	0.0490	0.0678	0.0331	0.0412	0.00126
13	share_internal	0.566	0.540	0.506	0.425	0.0496
14	n_unique_pages	610.	128.	215.	76.8	3.7
15	cluster_hc	1.02	2.58	2.38	1.92	4.64

```
# Definér 5 danske labels baseret på de opdaterede profiler
```

```
customer_df <- customer_df %>%
  mutate(cluster_label = case_when(
    cluster_km == 1 ~ "Power-brugeren (Mest mobil)",
    cluster_km == 2 ~ "Desktop-traditionalisten",
    cluster_km == 3 ~ "Veteranen (Høj historik)",
    cluster_km == 4 ~ "Mobil-nykommeren",
    cluster_km == 5 ~ "Spørgelses-brugeren (Lav aktivitet)",
    TRUE ~ "Andet"
  ))
```

```
# Tjek den nye fordeling
```

```
table(customer_df$cluster_label, customer_df$churn)
```

	0	1
Desktop-traditionalisten	250	180
Mobil-nykommeren	229	134
Power-brugeren (Mest mobil)	132	117
Spørgelses-brugeren (Lav aktivitet)	61	39
Veteranen (Høj historik)	68	65

```
# Gem mapping til din Tidymodels-fil
```

```
cluster_mapping <- customer_df %>%
  select(pseudo_id, cluster_label)
```

```
saveRDS(cluster_mapping, "data/cluster_mapping.rds")
```

```
# Tjek fordelingen
```

```
table(customer_df$cluster_label)
```

Desktop-traditionalisten	Mobil-nykommeren
430	363
Power-brugeren (Mest mobil)	Spørgelses-brugeren (Lav aktivitet)
249	100
Veteranen (Høj historik)	
133	

6.3 Churn model

```
pacman::p_load(
  tidyverse, tidymodels, themis, ranger, glmnet,
  kernlab, kkn, xgboost, naivebayes, rpart, future, vip
)
```

```

# Indlæs data
model_data <- readRDS("data/model_data.rds")
cluster_mapping <- readRDS("data/cluster_mapping.rds")

# Join cluster labels and clean initial data
model_data_clean <- model_data %>%
  # 1. Join the clusters
  left_join(cluster_mapping, by = "pseudo_id") %>%
  # 2. Convert label to factor for modeling
  mutate(cluster_label = as.factor(cluster_label)) %>%
  # 3. Rens data (fjern leakage og ID'er)
  select(
    -pseudo_id,
    -subscription_cancel_date,
    -type,
    -reason,
    -expiration_date,
    -order_trackertag,
    -first_campaign_day,
    -last_campaign_day,
    -continued_subscription
  ) %>%
  mutate(churn = factor(churn, levels = c("0", "1")))

# Check that cluster_label is present
glimpse(model_data_clean)

```

```

Rows: 1,275
Columns: 14
$ account_active_days      <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ koen                     <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ permission_given_order   <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today   <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, ~
$ previous_subscriptions    <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns        <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials           <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ newsletters_before_order  <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0,~
$ newsletters_after_order   <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0,~
$ churn                     <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0,~
$ early_churn               <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,~
$ age                       <dbl> 76.33699, 69.78630, 39.75342, 45.69041, 74.29~
$ kundetid_gruppe          <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~
$ cluster_label             <fct> Power-brugeren (Mest mobil), Power-brugeren (~

```

```

# Split
set.seed(8)
split <- initial_split(model_data_clean, prop = 0.8, strata = churn)
train_data <- training(split)
test_data <- testing(split)

# CV
folds <- vfold_cv(train_data, v = 5, strata = churn)

# Recipe

```

```

rec <- recipe(churn ~ ., data = train_data) %>%
  step_rm(early_churn) %>% # VERY IMPORTANT: avoid leakage
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_smote(churn)
# step_downsample(churn)

# Models
decision_tree_spec <- decision_tree(
  tree_depth = tune(),
  min_n = tune(),
  cost_complexity = tune()
) %>%
  set_engine("rpart") %>%
  set_mode("classification")

log_reg_spec <- logistic_reg(
  penalty = tune(),
  mixture = tune()
) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

rand_forest_spec <- rand_forest(
  mtry = tune(),
  min_n = tune(),
  trees = 500
) %>%
  set_engine("ranger", importance = "permutation") %>%
  set_mode("classification")

svm_rbf_spec <- svm_rbf(
  cost = tune(),
  rbf_sigma = tune()
) %>%
  set_engine("kernlab") %>%
  set_mode("classification")

xgb_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  mtry = tune()
) %>%
  set_engine("xgboost") %>%
  set_mode("classification")

# Workflow
workflow_set_obj <- workflow_set(
  preproc = list(rec = rec),
  models = list(
    decision_tree = decision_tree_spec,
    logistic_reg = log_reg_spec,

```



```

    rand_forest = rand_forest_spec,
    svm_rbf     = svm_rbf_spec,
    xgboost     = xgb_spec
  )
)

metrics <- metric_set(roc_auc, accuracy, f_meas, sens, spec)

# Train
plan(multisession)

set.seed(8)

# We use suppressMessages to hide the "Fold X: model Y/Z" notes
# and suppressWarnings to hide the Precision/Recall NA warnings.
results <- suppressMessages(suppressWarnings(
  workflow_set_obj %>%
    workflow_map(
      "tune_grid",
      resamples = folds,
      grid = 5,
      metrics = metrics,
      verbose = FALSE, # Switches off the tidymodels progress logger
      control = control_grid(
        save_pred = TRUE,
        save_workflow = TRUE
      )
    )
))

plan(sequential)

#
# plan(multisession)
#
# set.seed(8)
# results <- workflow_set_obj %>%
#   workflow_map(
#     "tune_grid",
#     resamples = folds,
#     grid = 5,
#     metrics = metrics,
#     control = control_grid(
#       verbose = TRUE,
#       save_pred = TRUE,
#       save_workflow = TRUE
#     )
#   )
#
# plan(sequential)
#

# Best model
best_model_id <- results %>%

```

```

rank_results(select_best = TRUE) %>%
filter(.metric == "f_meas") %>%
slice_max(mean, n = 1) %>%
pull(wflow_id)

best_tuned <- results %>%
  extract_workflow_set_result(best_model_id) %>%
  select_best(metric = "f_meas")

final_wf <- results %>%
  extract_workflow(best_model_id) %>%
  finalize_workflow(best_tuned)

# Final evaluation
final_fit <- final_wf %>%
  last_fit(split, metrics = metrics)

collect_metrics(final_fit)

```

```

# A tibble: 5 x 4
  .metric .estimator .estimate .config
  <chr>    <chr>         <dbl> <chr>
1 accuracy binary         0.788 pre0_mod0_post0
2 f_meas  binary         0.821 pre0_mod0_post0
3 sens    binary         0.838 pre0_mod0_post0
4 spec    binary         0.720 pre0_mod0_post0
5 roc_auc binary         0.869 pre0_mod0_post0

```

```

test_preds <- collect_predictions(final_fit)

test_preds %>%
  conf_mat(truth = churn, estimate = .pred_class)

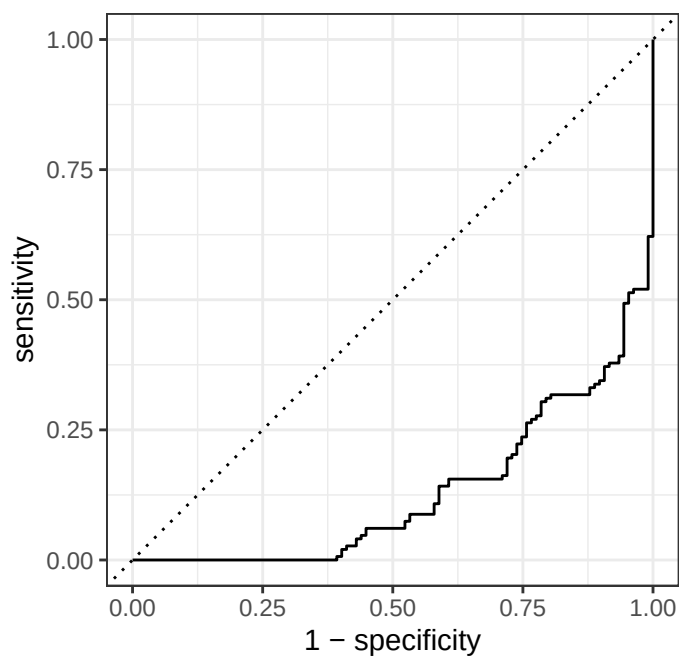
```

	Truth	
Prediction	0	1
0	124	30
1	24	77

```

test_preds %>%
  roc_curve(truth = churn, .pred_1) %>%
  autoplot()

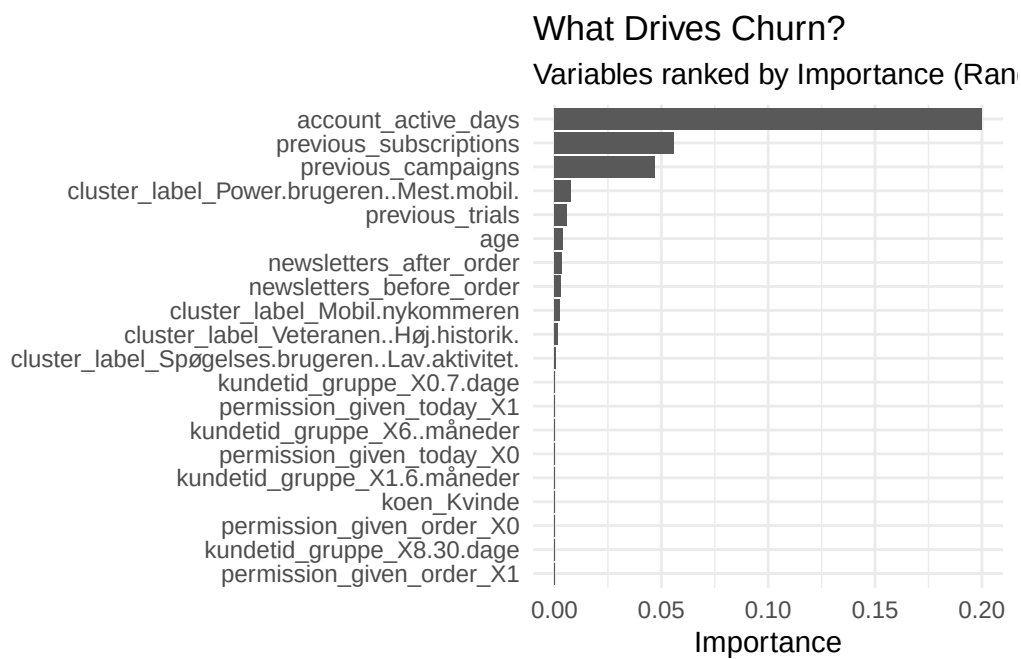
```



```
best_model_id
```

```
[1] "rec_rand_forest"
```

```
final_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 20) +
  theme_minimal() +
  labs(title = "What Drives Churn?",
       subtitle = "Variables ranked by Importance (Random Forest)")
```



```
#####
#  MODEL 2: EARLY CHURN (TASK 3)
#####

# Only customers who continued
early_data <- model_data_clean %>%
  filter(churn == "0") %>%
  mutate(early_churn = factor(early_churn, levels = c("0", "1")))

# Split
set.seed(8)
split_early <- initial_split(early_data, prop = 0.8, strata = early_churn)
train_early <- training(split_early)
test_early <- testing(split_early)

# CV
folds_early <- vfold_cv(train_early, v = 5, strata = early_churn)

# Recipe
rec_early <- recipe(early_churn ~ ., data = train_early) %>%
  step_rm(churn) %>% # remove parent target
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())
# step_downsample(early_churn)

# Workflow (reuse same models!)
workflow_set_early <- workflow_set(
  preproc = list(rec = rec_early),
  models = list(
    decision_tree = decision_tree_spec,
    logistic_reg = log_reg_spec,
    rand_forest = rand_forest_spec,
    svm_rbf = svm_rbf_spec,
    xgboost = xgb_spec
  )
)

# Train
plan(multisession)

set.seed(8)
results_early <- workflow_set_early %>%
  workflow_map(
    "tune_grid",
    resamples = folds_early,
    grid = 5,
    metrics = metrics
  )

```

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

i Creating pre-processing data to finalize 1 unknown parameter: "mtry"

→ A | warning: ! 23 columns were requested but there were 22 predictors in the data.
22 predictors will be used.

maximum number of iterations reached 0.0001029682 0.0001028812maximum number of iterations reached 0.00

i Creating pre-processing data to finalize 1 unknown parameter: "mtry"

```
plan(sequential)
```

```
# Best model
```

```
best_model_id_early <- results_early %>%  
  rank_results(select_best = TRUE) %>%  
  filter(.metric == "f_meas") %>%  
  slice_max(mean, n = 1) %>%  
  pull(wflow_id)
```

```
best_tuned_early <- results_early %>%  
  extract_workflow_set_result(best_model_id_early) %>%  
  select_best(metric = "f_meas")
```

```
final_wf_early <- results_early %>%  
  extract_workflow(best_model_id_early) %>%  
  finalize_workflow(best_tuned_early)
```

```
# Final evaluation
```

```
final_fit_early <- final_wf_early %>%  
  last_fit(split_early, metrics = metrics)
```

```
collect_metrics(final_fit_early)
```

```
# A tibble: 5 x 4
```

	.metric	.estimator	.estimate	.config
	<chr>	<chr>	<dbl>	<chr>
1	accuracy	binary	0.799	pre0_mod0_post0
2	f_meas	binary	0.871	pre0_mod0_post0
3	sens	binary	1	pre0_mod0_post0
4	spec	binary	0.375	pre0_mod0_post0
5	roc_auc	binary	0.899	pre0_mod0_post0

```
test_preds_early <- collect_predictions(final_fit_early)

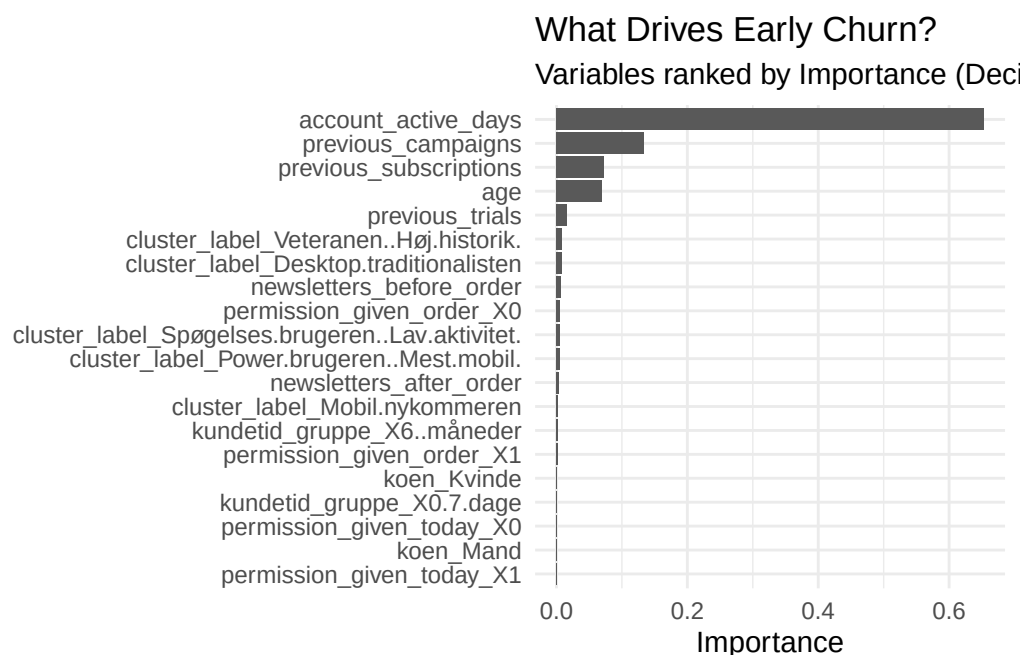
test_preds_early %>%
  conf_mat(truth = early_churn, estimate = .pred_class)
```

```
      Truth
Prediction 0  1
0 101 30
1   0 18
```

```
best_model_id_early
```

```
[1] "rec_xgboost"
```

```
final_fit_early %>%
  extract_fit_parsnip() %>%
  vip(num_features = 20) +
  theme_minimal() +
  labs(title = "What Drives Early Churn?",
       subtitle = "Variables ranked by Importance (Decision Tree)")
```



```
#####
# RESULTS VISUALIZATION & COMPARISON
#####

# 1. Show the "Leaderboard" for Model 1 (Churn)
# This lists all models ranked by F-Measure
cat("\n--- Model 1: Churn Leaderboard ---\n")
```

```
--- Model 1: Churn Leaderboard ---
```

```

results %>%
  rank_results(rank_metric = "f_meas", select_best = TRUE) %>%
  select(wflow_id, .metric, mean, std_err, rank) %>%
  filter(.metric == "f_meas") %>%
  print()

```

```

# A tibble: 5 x 5
  wflow_id      .metric mean std_err rank
  <chr>         <chr>   <dbl>   <dbl> <int>
1 rec_decision_tree f_meas  0.805 0.00164     1
2 rec_rand_forest   f_meas  0.792 0.0156     2
3 rec_xgboost       f_meas  0.773 0.0160     3
4 rec_logistic_reg  f_meas  0.735 0         4
5 rec_svm_rbf       f_meas  0.723 0.00668     5

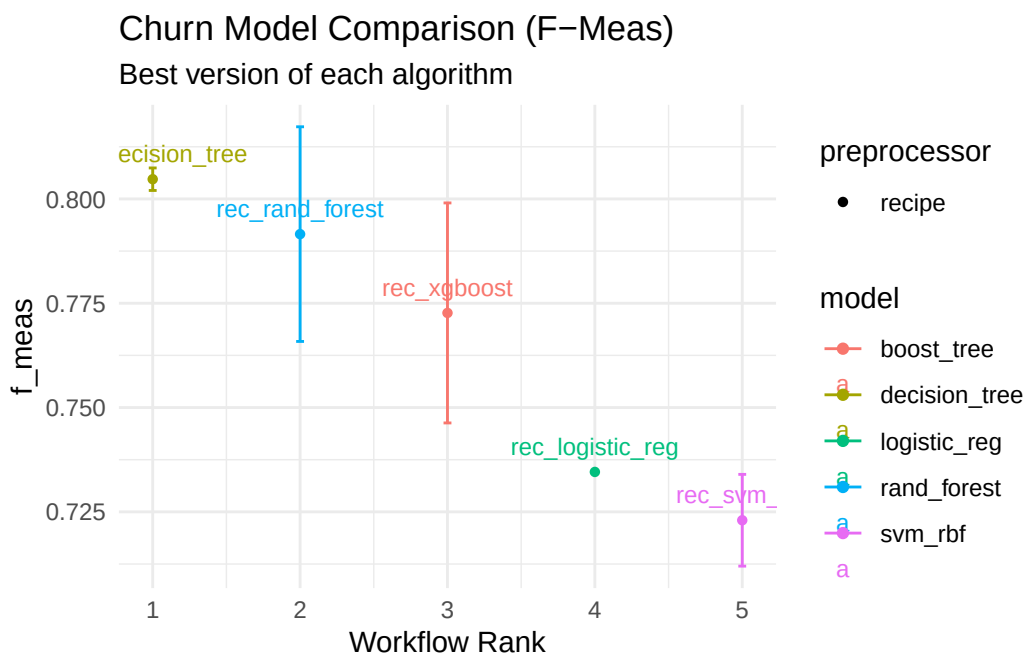
```

```

# 2. Plot Model 1 Comparison
# This gives you a visual look at how the different engines compared
p1 <- autoplot(results, metric = "f_meas", select_best = TRUE) +
  geom_text(aes(label = wflow_id), vjust = -1, size = 3) +
  labs(title = "Churn Model Comparison (F-Meas)",
       subtitle = "Best version of each algorithm") +
  theme_minimal()

print(p1)

```



```

# 3. Show the "Leaderboard" for Model 2 (Early Churn)
cat("\n--- Model 2: Early Churn Leaderboard ---\n")

```

```

--- Model 2: Early Churn Leaderboard ---

```

```

results_early %>%
  rank_results(rank_metric = "f_meas", select_best = TRUE) %>%
  select(wflow_id, .metric, mean, std_err, rank) %>%
  filter(.metric == "f_meas") %>%
  print()

```

```

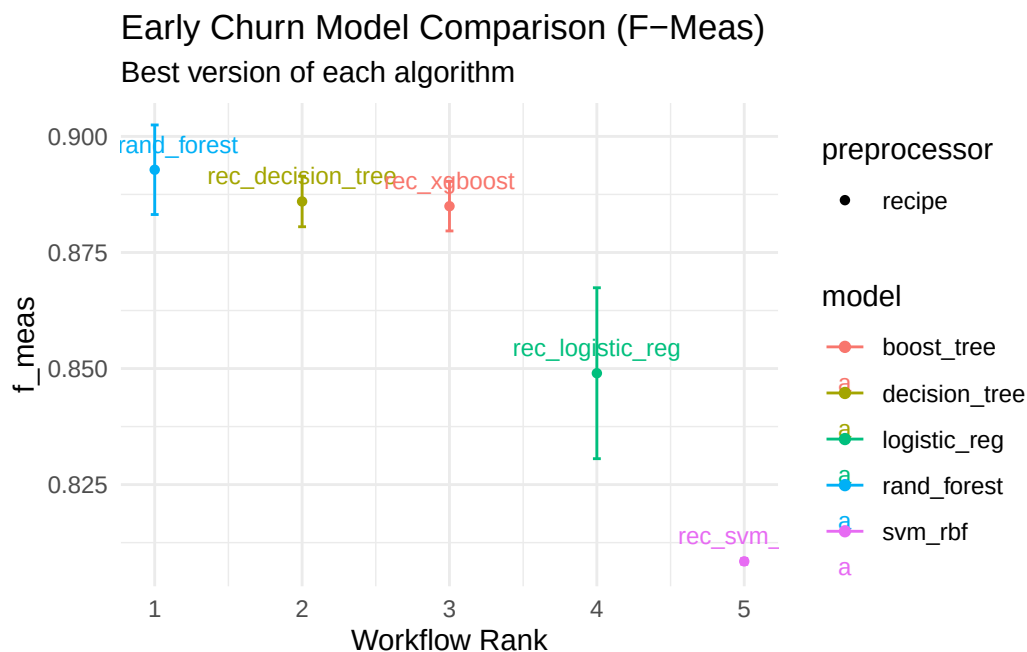
# A tibble: 5 x 5
  wflow_id      .metric mean  std_err rank
  <chr>         <chr>   <dbl>   <dbl> <int>
1 rec_rand_forest f_meas  0.893 0.00586     1
2 rec_decision_tree f_meas  0.886 0.00331     2
3 rec_xgboost      f_meas  0.885 0.00324     3
4 rec_logistic_reg f_meas  0.849 0.0112     4
5 rec_svm_rbf       f_meas  0.808 0.000384    5

```

```

# 4. Plot Model 2 Comparison
p2 <- autoplot(results_early, metric = "f_meas", select_best = TRUE) +
  geom_text(aes(label = wflow_id, vjust = -1, size = 3) +
    labs(title = "Early Churn Model Comparison (F-Meas)",
      subtitle = "Best version of each algorithm") +
    theme_minimal()
print(p2)

```



```

# 5. Identify exactly which Hyperparameters the winner used
cat("\n--- Best Model Hyperparameters ---\n")

```

```

--- Best Model Hyperparameters ---

```



```
cat("Churn Model:", best_model_id, "\n")
```

Churn Model: rec_rand_forest

```
print(best_tuned)
```

```
# A tibble: 1 x 3
  mtry min_n .config
  <int> <int> <chr>
1     23     21 pre0_mod5_post0
```

```
cat("\nEarly Churn Model:", best_model_id_early, "\n")
```

Early Churn Model: rec_xgboost

```
print(best_tuned_early)
```

```
# A tibble: 1 x 5
  mtry trees tree_depth learn_rate .config
  <int> <int>      <int>      <dbl> <chr>
1     17    500          11      0.001 pre0_mod4_post0
```

```
#
# # 1. Get predictions for all users
# final_results <- final_fit %>%
#   extract_workflow() %>%
#   augment(model_data_clean)
#
# # 2. Create the Cluster-Risk Matrix
# risk_profile <- final_results %>%
#   group_by(cluster_label) %>%
#   summarise(
#     n_users = n(),
#     # Model's average predicted probability of churn
#     avg_predicted_risk = mean(.pred_1),
#     # Actual churn recorded in data
#     actual_churn_rate = mean(churn == "1"),
#     # Early Churn rate within this cluster
#     early_churn_rate = mean(early_churn == 1),
#     # Loyalty score (How many stayed)
#     loyalty_rate = mean(churn == "0")
#   ) %>%
#   arrange(desc(avg_predicted_risk))
#
# print(risk_profile)
```

6.4 Neural network churn model

Åbn <https://colab.research.google.com/>, lav "New Notebook" og importer JP_NN2.ipynb. Derefter skal subscription_v2.csv og cancellation.csv indsættes i Filer under "sample_data". Derefter kan koden køres.

Vi har her en rensningsblok og derefter fire modeller der i alt fylder ~18000 tegn inkl. mellemrum.

Model 01 fungerer som en baseline og er implementeret som et simpelt neuralt netværk ved brug af Keras Sequential API. I koden ses dette bl.a. ved et enkelt skjult lag (`Dense(..., activation='relu')`) med L2-regularisering samt et outputlag med sigmoid-aktivering. Modellen kompiles med adam og `binary_crossentropy`. Formålet er at etablere et sammenligningsgrundlag uden avancerede teknikker.

Model 02 udvider baseline-modellen ved at inkludere flere lag, eksempelvis `Dense(64)` og `Dense(32)`, samt `BatchNormalization()`. Dette øger modellens evne til at lære komplekse mønstre og stabiliserer træningen. Modellen er dermed mere kompleks end Model 01.

Model 03 fokuserer på håndtering af ubalancerede data ved brug af SMOTE, som i koden ses ved `SMOTE().fit_resample(X_train, y_train)`. Dette skaber et balanceret datasæt ved at generere syntetiske observationer af minoritetsklassen. Selve netværksarkitekturen ligner tidligere modeller, men træningen sker på forbedret datagrundlag.

Model 04 anvender SMOTENC (`SMOTENC(categorical_features=...)`), som tager højde for både numeriske og kategoriske variabler ved oversampling. Dette giver mere realistiske syntetiske data sammenlignet med standard SMOTE. Modellen kombinerer dermed en neural netværksarkitektur med en mere avanceret metode til håndtering af datasæt-ubalance.

Til sidst vurderer vi modellerne mod hinanden, laver en ROC kurve for dem alle, og sammenligner confusion matrixe for model 03 og model 04.

— Model Performance Leaderboard — Model AUC Recall (Churn Caught) Accuracy Model 03 (SMOTE) 0.857 0.679 0.760 Model 04 (SMOTENC) 0.828 0.698 0.744 Model 02 (Complex) 0.819 0.679 0.736 Model 01 (Baseline) 0.817 0.689 0.724

Resultaterne viser, at Model 03 (SMOTE) opnår den højeste AUC (0.857) og accuracy (0.760), hvilket indikerer den bedste samlede performance. Model 04 (SMOTENC) har den højeste recall (0.698) og er dermed bedst til at identificere churn-kunder.

Model 02 (Complex) og Model 01 (Baseline) præsterer relativt ens, hvilket tyder på, at øget modelkompleksitet alene ikke giver en væsentlig forbedring. Samtidig ses det, at modeller med SMOTE/SMOTENC generelt performer bedre, hvilket understreger vigtigheden af at håndtere datasæt-ubalance.

Samlet set afhænger valget af model af, om der prioriteres høj recall (Model 04) eller samlet performance (Model 03).